Activity 4 Report: Database Users, Roles, Privileges and Access Control

COMP 018 Database Administration | Project tested: BrewPOS (Cafe Point-of-Sale) (Spring Boot + PostgreSQL) | Date of testing: 6 November 2026

1. Cover and links

Item	Value
Testing group	G08: Borreta, Joshua; Bola, Martin Lawrence; Juatchon, Joaquin Phillip; Marcos, Karl Raphael; Pacheco Jr., Sancho; Vera Cruz, Euan Joseph
Project tested	BrewPOS (Cafe Point-of-Sale)
Owning group	Group 4: Olivarez, Charles Kean E.; Infantado, Gerwin G.; Pocsidio, Jhon Louie F.; Abangan, SJ Kaizen M.; Bartolata, Redge Raindel N.
Repository	https://github.com/Nycri/brewpos
Tracker	https://github.com/EclipseChan/bwisIT-Solutions---Campus-Canteen-Pre-Order-and-Queue-Management-System/tree/main/other_group_tests
Evidence files	a4_access_tests.txt, a3_deploy.txt, a3_app.txt

This is a full re-test. The database was dropped and rebuilt from the repository files, and every result below comes from this run.

2. Access-control plan (from the project docs)

docs/user-roles.md describes three application roles (Store Manager, Cashier, Viewer/Auditor) and four database roles: a runtime login account brewpos_app, and group roles app_runtime_grp, app_readonly_grp and app_admin_grp. database/05_create_app_role.sql creates brewpos_app. The group roles exist only as SQL in the document (section 4). The README says end users have no login at all and that authentication is out of scope.

3. The documented script, run as written

05_create_app_role.sql was run on the fresh database, the password was set, and the script was run a second time to check it can be repeated.

Check	Result
Role attributes	LOGIN only. Not superuser, no CREATEDB, no CREATEROLE, no replication, no bypass RLS, NOINHERIT. Password stored as SCRAM-SHA-256
Session defaults	search_path=public, statement_timeout=30s, lock_timeout=10s. A pg_sleep(31) test was cancelled by the timeout
Second run	Harmless (grants repeated, no errors)
Privilege matrix	staff: S. menu_category and menu_item: S, I, U, D. order_ticket and order_line_item: S, I. No TRUNCATE

	anywhere. Sequences: usable. CREATE on schema public: no
Connect to the postgres maintenance database	Denied (FATAL, no CONNECT privilege)

4. Tests as brewpos_app (real login)

Test	Expected	Actual
Read staff	Allowed	Allowed
INSERT or UPDATE staff	Denied	Denied (permission denied for table staff)
INSERT an order (rolled back)	Allowed	Allowed
UPDATE or DELETE order_ticket	Denied	Denied
UPDATE sold_price or DELETE in order_line_item	Denied	Denied
TRUNCATE order_line_item	Denied	Denied
UPDATE menu_item price (rolled back)	Allowed	Allowed
DELETE menu_item that was never sold (rolled back)	Allowed (script comment says so)	Allowed
DELETE a category that still has items	Blocked by the foreign key	Blocked
CREATE TABLE, DROP TABLE, ALTER TABLE	Denied	Denied (no CREATE on schema, not owner)
CREATE ROLE, read pg_authid	Denied	Denied
VACUUM a table	Denied (not owner)	Skipped with a warning

5. The group roles from docs/user-roles.md, applied

The SQL in section 4 of the document was run as written on the test database.

Finding	Result
Is brewpos_app a member of app_runtime_grp? (the document says it is)	No. pg_auth_members has no row for it. The two sets of privileges are separate copies
app_readonly_grp	Reads all 5 tables. Cannot insert and cannot use the sequences
app_runtime_grp	Inserts orders, edits the catalog (including delete of categories). Cannot update or delete orders
app_admin_grp	Edits the catalog. Cannot insert orders. Has TRUNCATE on menu_category and menu_item because the script grants ALL PRIVILEGES. A TRUNCATE ... CASCADE was stopped only because the role has no rights on order_line_item
Privilege counts after the script	app_admin_grp 17, app_readonly_grp 5, app_runtime_grp 13, brewpos_app 13

6. The application running as brewpos_app

The app was restarted with brewpos_app in .env. pg_stat_activity showed all 10 pooled connections as brewpos_app. Reading the menu, adding a category and an item, changing the price, placing an order and loading Daily Sales all worked. Deleting a category with items returned HTTP 409. The README's recommended least-privilege setup therefore works for every feature.

7. Who can do what at the API

There is no login, so the application roles are only visible in the front end. These calls carried no cookie, token or password:

Call	What a Cashier should be able to do	Result

POST /api/menu-items (add an item)	Nothing (Menu is read-only for cashiers)	HTTP 201, item created
PUT /api/menu-items/{id} (change a price)	Nothing	HTTP 200 (done earlier as brewpos_app)
POST /api/orders with staffId of the Manager	Only record orders as himself	HTTP 201; the order is stored under Demo Manager
GET /api/staff	Not specified	HTTP 200, full staff list returned

There is no Spring Security in pom.xml and the only role check in the source is the dropdown in app.js and index.html. The database account is the real security boundary, and it is shared by everyone who uses the app. pg_hba.conf uses scram-sha-256 on all rules, with no trust entries.

8. Issues and recommendations

#	Finding	Recommendation
1	No authentication or authorisation at the API. The Cashier and Manager roles are cosmetic, and the staffId on an order is taken from the client	Add login (Spring Security) and take the staff id from the session. At minimum protect the menu endpoints
2	The README tells users to connect as postgres first. The least-privilege account exists but is optional	Make brewpos_app the default in .env.example
3	docs/user-roles.md says brewpos_app is a member of app_runtime_grp, and it is not. The group roles are not in any script	Add a script (06 or 07) that creates the groups and grants membership, and keep one set of grants
4	app_admin_grp is given ALL PRIVILEGES on the menu tables, which includes TRUNCATE, TRIGGER and REFERENCES	Grant SELECT, INSERT, UPDATE, DELETE only
5	brewpos_app can DELETE menu items and categories. Items never sold can be removed, which the script comment accepts for test cleanup	Revoke DELETE on menu_item for production, as the script comment suggests
6	The database lets an order exist with no lines (the INSERT into order_ticket alone succeeds). The API refuses it, the database does not	Add a deferred check or build orders only through one transaction (the app already does)
7	No audit trail of who changed prices or added items	Add a small log table or trigger on menu_item

9. Feedback for the owners

Findings 1 to 4 are the most useful to act on. The role script itself worked correctly in every test.

10. Limitations

Group role tests used SET ROLE from the administrator account, because those roles cannot log in. The roles app_runtime_grp, app_readonly_grp and app_admin_grp are new in this cluster. app_runtime_grp and app_readonly_grp already existed from another project tested earlier on the same server, so their privileges here were set by the BrewPOS SQL only on this database.

AI usage declaration

- Tool: Claude (Anthropic).

- Tasks: interpreting the results of the testing.
- Validation: the commands were run by the group on its own computer, and every result in this report comes from the saved output files.